

DEPLOYMENT

Claude Code on Amazon Bedrock

Learn about configuring Claude Code through Amazon Bedrock, including setup, IAM configuration, and troubleshooting.

Deploying Claude Code across your organization? Talk to sales about enterprise plans, SSO, and centralized billing.

[View plans](#)
[Contact sales →](#)

Prerequisites

Before configuring Claude Code with Bedrock, ensure you have:

- An AWS account with Bedrock access enabled
- Access to desired Claude models (for example, Claude Sonnet 4.6) in Bedrock
- AWS CLI installed and configured (optional - only needed if you don't have another mechanism for getting credentials)
- Appropriate IAM permissions

To sign in with your own Bedrock credentials, follow [Sign in with Bedrock](#) below. To deploy Claude Code across a team, use the [manual setup](#) steps and [pin your model versions](#) before rolling out.

Sign in with Bedrock

If you have AWS credentials and want to start using Claude Code through Bedrock, the login wizard walks you through it. You complete the AWS-side prerequisites once per account; the wizard handles the Claude Code side.

1 Enable Anthropic models in your AWS account

In the [Amazon Bedrock console](#), open the Model catalog, select an Anthropic model, and submit the use case form. Access is granted immediately after submission. See [Submit use](#)

case details for AWS Organizations and IAM configuration for the permissions your role needs.

2 Start Claude Code and choose Bedrock

Run `cclaude`. At the login prompt, select **3rd-party platform**, then **Amazon Bedrock**.

3 Follow the wizard prompts

Choose how you authenticate to AWS: an AWS profile detected from your `~/.aws` directory, a Bedrock API key, an access key and secret, or credentials already in your environment. The wizard picks up your region, verifies which Claude models your account can invoke, and lets you pin them. It saves the result to the `env` block of your user settings file, so you don't need to export environment variables yourself.

After you've signed in, run `/setup-bedrock` any time to reopen the wizard and change your credentials, region, or model pins.

Set up manually

To configure Bedrock through environment variables instead of the wizard, for example in CI or a scripted enterprise rollout, follow the steps below.

1. Submit use case details

First-time users of Anthropic models are required to submit use case details before invoking a model. This is done once per AWS account.

1. Ensure you have the right IAM permissions described below
2. Navigate to the Amazon Bedrock console
3. Select an Anthropic model from the **Model catalog**
4. Complete the use case form. Access is granted immediately after submission.

If you use AWS Organizations, you can submit the form once from the management account using the PutUseCaseForModelAccess API. This call requires the `bedrock:PutUseCaseForModelAccess` IAM permission. Approval extends to child accounts automatically.

2. Configure AWS credentials

Claude Code uses the default AWS SDK credential chain. Set up your credentials using one of these methods:

Option A: AWS CLI configuration

```
aws configure
```

Option B: Environment variables (access key)

```
export AWS_ACCESS_KEY_ID=your-access-key-id
export AWS_SECRET_ACCESS_KEY=your-secret-access-key
export AWS_SESSION_TOKEN=your-session-token
```

Option C: Environment variables (SSO profile)

```
aws sso login --profile=<your-profile-name>

export AWS_PROFILE=your-profile-name
```

Option D: AWS Management Console credentials

```
aws login
```

[Learn more](#) about `aws login` .

Option E: Bedrock API keys

```
export AWS_BEARER_TOKEN_BEDROCK=your-bedrock-api-key
```

Bedrock API keys provide a simpler authentication method without needing full AWS credentials.

[Learn more about Bedrock API keys.](#)

Advanced credential configuration

Claude Code supports automatic credential refresh for AWS SSO and corporate identity providers.

Add these settings to your Claude Code settings file (see [Settings](#) for file locations).

These two settings have different trigger conditions:

- `awsAuthRefresh` : runs only when Claude Code detects that your AWS credentials are expired, either locally based on their timestamp or when Bedrock returns a credential error, then retries the request with refreshed credentials.
- `awsCredentialExport` : runs at session start and on each credential reload, even when the credentials in your AWS default credential provider chain are still valid. Use this when your Bedrock account requires cross-account credentials that differ from the ones the default provider chain would resolve.

Example configuration

```
{
  "awsAuthRefresh": "aws sso login --profile myprofile",
  "env": {
    "AWS_PROFILE": "myprofile"
  }
}
```

Configuration settings explained

`awsAuthRefresh` : Use this for commands that modify the `.aws` directory, such as updating credentials, SSO cache, or config files. The command's output is displayed to the user, but interactive input isn't supported. This works well for browser-based SSO flows where the CLI displays a URL or code and you complete authentication in the browser.

`awsCredentialExport` : Only use this if you can't modify `.aws` and must directly return credentials. This command runs whenever credentials need to be refreshed, not only when credentials are expired. Output is captured silently and not shown to the user. The command must output JSON in this format:

```
{
  "Credentials": {
    "AccessKeyId": "value",
    "SecretAccessKey": "value",
    "SessionToken": "value",
    "Expiration": "2026-01-01T00:00:00Z"
  }
}
```

As of Claude Code v2.1.181, the flat output from `aws configure export-credentials --format process` is also accepted, with the same keys at the top level instead of nested under `Credentials`.

`Expiration` is optional. As of Claude Code v2.1.176, when the command returns a valid ISO 8601 `Expiration`, Claude Code caches the credentials until five minutes before that time. Without it, or on earlier versions, credentials are cached for one hour.

3. Configure Claude Code

Set the following environment variables to enable Bedrock:

```
# Enable Bedrock integration
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1 # optional if your AWS profile
already sets a region

# Optional: Override the AWS region for the small/fast model
(Bedrock and Mantle).
# On Bedrock, has no effect without ANTHROPIC_DEFAULT_HAIKU_MODEL
# or the deprecated ANTHROPIC_SMALL_FAST_MODEL set.
export ANTHROPIC_SMALL_FAST_MODEL_AWS_REGION=us-west-2

# Optional: Override the Bedrock endpoint URL for custom
endpoints or gateways
# export ANTHROPIC_BEDROCK_BASE_URL=https://bedrock-runtime.us-
east-1.amazonaws.com
```

When enabling Bedrock for Claude Code, keep the following in mind:

- As of v2.1.172, you only need to set `AWS_REGION` to override your AWS profile's region or when

your profile has no region. Claude Code resolves the region in this order:

- `AWS_REGION`
- `AWS_DEFAULT_REGION`
- the `region` set on your active AWS profile, read from the AWS shared credentials file first and then the shared config file, matching AWS SDK precedence
- `us-east-1`

The active profile is `AWS_PROFILE` if set, otherwise `default`. Set `AWS_SHARED_CREDENTIALS_FILE` or `AWS_CONFIG_FILE` to point at non-default file paths. Run `/status` to see the resolved region. When the region came from your AWS config files or the default fallback, `/status` also notes the source. On v2.1.171 and earlier, Claude Code does not read the AWS config files, so set `AWS_REGION` explicitly.

- When using Bedrock, the `/logout` command is unavailable since authentication is handled through AWS credentials.
- The WebSearch tool is not available on Bedrock. See [WebSearch tool behavior](#).
- You can use settings files for environment variables like `AWS_PROFILE` that you don't want to leak to other processes. See [Settings](#) for more information.

4. Pin model versions

⚠ Pin specific model versions when deploying to multiple users. Without pinning, model aliases such as `sonnet` and `opus` resolve to Claude Code's built-in default for Bedrock, which can lag the newest release and may not yet be available in your account. Claude Code **falls back** to the previous version at startup when the default is unavailable, but pinning lets you control when your users move to a new model.

Set these environment variables to specific Bedrock model IDs.

Without `ANTHROPIC_DEFAULT_OPUS_MODEL`, the `opus` alias on Bedrock resolves to Opus 4.6. Set it to the Opus 4.8 ID to use the latest model:

```
export ANTHROPIC_DEFAULT_OPUS_MODEL='us.anthropic.claude-opus-4-8'
export ANTHROPIC_DEFAULT_SONNET_MODEL='us.anthropic.claude-sonnet-4-6'
export ANTHROPIC_DEFAULT_HAIKU_MODEL='us.anthropic.claude-haiku-4-5-20251001-v1:0'
```

These variables use cross-region inference profile IDs (with the `us.` prefix). If you use a different region prefix or application inference profiles, adjust accordingly. In AWS GovCloud regions, use the `us-gov.` prefix. For current and legacy model IDs, see [Models overview](#). See [Model configuration](#) for the full list of environment variables.

Claude Code uses these default models when no pinning variables are set:

Model type	Default value
Primary model	<code>us.anthropic.claude-sonnet-4-5-20250929-v1:0</code>
Small/fast model	Same as primary model

Background tasks such as session title generation use the small/fast model, normally a Haiku-class model. On Bedrock, Claude Code defaults this to the primary model because Haiku may not be enabled in every account or region. To use Haiku for background tasks, set `ANTHROPIC_DEFAULT_HAIKU_MODEL` to a model ID that is available in your account.

To customize models further, use one of these methods:

```
# Using inference profile ID
export ANTHROPIC_MODEL='us.anthropic.claude-sonnet-4-6'
export ANTHROPIC_DEFAULT_HAIKU_MODEL='us.anthropic.claude-haiku-4-5-20251001-v1:0'

# Using application inference profile ARN
export ANTHROPIC_MODEL='arn:aws:bedrock:us-east-2:your-account-id:application-inference-profile/your-model-id'

# Optional: Disable prompt caching if needed
export DISABLE_PROMPT_CACHING=1

# Optional: Request 1-hour prompt cache TTL instead of the 5-minute default
export ENABLE_PROMPT_CACHING_1H=1
```

The 1-hour cache TTL is billed at a higher rate than the 5-minute default. See [cache lifetime](#).

❗ Prompt caching may not be available in all Bedrock regions. If cache token counts stay at zero, check [supported models, regions, and limits](#) in the Bedrock documentation.

Map each model version to an inference profile

The `ANTHROPIC_DEFAULT*_MODEL` environment variables configure one inference profile per model family. If your organization needs to expose several versions of the same family in the `/model` picker, each routed to its own application inference profile ARN, use the `modelOverrides` setting in your **settings file** instead.

This example maps four Opus versions to distinct ARNs so users can switch between them without bypassing your organization's inference profiles:

```
{
  "modelOverrides": {
    "claude-opus-4-7": "arn:aws:bedrock:us-east-2:123456789012:application-inference-profile/opus-47-prod",
    "claude-opus-4-6": "arn:aws:bedrock:us-east-2:123456789012:application-inference-profile/opus-46-prod",
    "claude-opus-4-5-20251101": "arn:aws:bedrock:us-east-2:123456789012:application-inference-profile/opus-45-prod",
    "claude-opus-4-1-20250805": "arn:aws:bedrock:us-east-2:123456789012:application-inference-profile/opus-41-prod"
  }
}
```

When a user selects one of these versions in `/model`, Claude Code calls Bedrock with the mapped ARN. Versions without an override fall back to the built-in Bedrock model ID or any matching inference profile discovered at startup. See **Override model IDs per version** for details on how overrides interact with `availableModels` and other model settings.

Startup model checks

When Claude Code starts with Bedrock configured, it verifies that the models it intends to use are accessible in your account. This check requires Claude Code v2.1.94 or later.

If you have pinned a model version that is older than the current Claude Code default, and your account can invoke the newer version, Claude Code prompts you to update the pin. Accepting writes the new model ID to your **user settings file** and restarts Claude Code. Declining is remembered until the next default version change. Pins that point to an **application inference profile ARN** are skipped, since those are managed by your administrator.

If you have not pinned a model and the current default is unavailable in your account, Claude Code

falls back to the previous version for the current session and shows a notice. The fallback is not persisted. Enable the newer model in your Bedrock account or **pin a version** to make the choice permanent.

IAM configuration

Create an IAM policy with the required permissions for Claude Code:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowModelAndInferenceProfileAccess",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream",
        "bedrock:ListInferenceProfiles",
        "bedrock:GetInferenceProfile"
      ],
      "Resource": [
        "arn:aws:bedrock:*:inference-profile/*",
        "arn:aws:bedrock:*:application-inference-profile/*",
        "arn:aws:bedrock:*:foundation-model/*"
      ]
    },
    {
      "Sid": "AllowMarketplaceSubscription",
      "Effect": "Allow",
      "Action": [
        "aws-marketplace:ViewSubscriptions",
        "aws-marketplace:Subscribe"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:CalledViaLast": "bedrock.amazonaws.com"
        }
      }
    }
  ]
}
```

For more restrictive permissions, you can limit the Resource to specific inference profile ARNs.

`bedrock:GetInferenceProfile` lets Claude Code resolve an **application inference profile ARN** to its backing foundation model, which is used to select the correct request shape for that model.

If the token is missing this permission, Claude Code recovers automatically by retrying once with the alternate shape, so requests still succeed but each new model adds an extra round-trip. Granting the permission avoids the retry. This applies most often to `AWS_BEARER_TOKEN_BEDROCK` deployments, where the token's policy is typically narrower than a full IAM role.

For details, see **Bedrock IAM documentation**.

❗ Create a dedicated AWS account for Claude Code to simplify cost tracking and access control.

1M token context window

Claude Opus 4.6 and later, and Sonnet 4.6, support the **1M token context window** on Amazon Bedrock. Claude Code automatically enables the extended context window when you select a 1M model variant.

The **setup wizard** offers a 1M context option when it pins models. To enable it for a manually pinned model instead, append `[1m]` to the model ID. See **Pin models for third-party deployments** for details.

Service tiers

Amazon Bedrock service tiers let you trade off cost against latency. Set

`ANTHROPIC_BEDROCK_SERVICE_TIER` to `default`, `flex`, or `priority`:

```
export ANTHROPIC_BEDROCK_SERVICE_TIER=priority
```

Claude Code sends this as the `X-Amzn-Bedrock-Service-Tier` header on each request. Tier availability varies by model and region. Reserved capacity uses a **provisioned throughput** ARN as the model ID instead of this setting.

AWS Guardrails

Amazon Bedrock Guardrails let you implement content filtering for Claude Code. Create a Guardrail in the **Amazon Bedrock console**, publish a version, then add the Guardrail headers to your **settings file**. Enable Cross-Region inference on your Guardrail if you're using cross-region inference profiles.

Example configuration:

```
{
  "env": {
    "ANTHROPIC_CUSTOM_HEADERS": "X-Amzn-Bedrock-
GuardrailIdentifier: your-guardrail-id\nX-Amzn-Bedrock-
GuardrailVersion: 1"
  }
}
```

Use the Mantle endpoint

Mantle is an Amazon Bedrock endpoint that serves Claude models through the native Anthropic API shape rather than the Bedrock Invoke API. It uses the same AWS credentials, IAM permissions, and `awsAuthRefresh` configuration described earlier on this page.

❗ Mantle requires Claude Code v2.1.94 or later. Run `claude --version` to check.

Enable Mantle

With AWS credentials already configured, set `CLAUDE_CODE_USE_MANTLE` to route requests to the Mantle endpoint:

```
export CLAUDE_CODE_USE_MANTLE=1
export AWS_REGION=us-east-1
```

Claude Code constructs the endpoint URL from the AWS region. As of v2.1.172, the region is resolved with the same precedence as **Bedrock above**; earlier versions use `AWS_REGION` only. To override the URL for a custom endpoint or gateway, set `ANTHROPIC_BEDROCK_MANTLE_BASE_URL`.

Run `/status` inside Claude Code to confirm. The provider line shows `Amazon Bedrock (Mantle)` when Mantle is active.

Select a Mantle model

Mantle uses model IDs prefixed with `anthropic.` and without a version suffix, for example `anthropic.claude-haiku-4-5`. The models available to your account depend on what your organization has been granted; additional model IDs are listed in your onboarding materials from AWS. Contact your AWS account team to request access to allowlisted models.

Set the model with the `--model` flag or with `/model` inside Claude Code:

```
claude --model anthropic.claude-haiku-4-5
```

Run Mantle alongside the Invoke API

The models available to you on Mantle may not include every model you use today. Setting both `CLAUDE_CODE_USE_BEDROCK` and `CLAUDE_CODE_USE_MANTLE` lets Claude Code call both endpoints from the same session. Model IDs that match the Mantle format are routed to Mantle, and all other model IDs go to the Bedrock Invoke API.

```
export CLAUDE_CODE_USE_BEDROCK=1
export CLAUDE_CODE_USE_MANTLE=1
```

To surface a Mantle model in the `/model` picker, list its ID in `availableModels` in your settings file. This setting also restricts the picker to the listed entries. Listing `anthropic.claude-haiku-4-5` removes the bare `haiku` alias from the picker, so also list version prefixes or full IDs for the versions you want to keep selectable. The Mantle ID and the `haiku` alias resolve to the same model family, so the merge keeps only the more specific entry. See Merge behavior:

```
{
  "availableModels": ["opus", "sonnet", "claude-haiku-4-5",
    "anthropic.claude-haiku-4-5"]
}
```

Entries with the `anthropic.` prefix are added as custom picker options and routed to Mantle. Replace `anthropic.claude-haiku-4-5` with the model ID your account has been granted. See Restrict model selection for how `availableModels` interacts with other model settings.

When both providers are active, `/status` shows `Amazon Bedrock + Amazon Bedrock (Mantle)`.

Route Mantle through a gateway

If your organization routes model traffic through a centralized **LLM gateway** that injects AWS credentials server-side, disable client-side authentication so Claude Code sends requests without SigV4 signatures or `x-api-key` headers:

```
export CLAUDE_CODE_USE_MANTLE=1
export CLAUDE_CODE_SKIP_MANTLE_AUTH=1
export ANTHROPIC_BEDROCK_MANTLE_BASE_URL=https://your-gateway.example.com
```

Mantle environment variables

These variables are specific to the Mantle endpoint. See **Environment variables** for the full list.

Variable	Purpose
CLAUDE_CODE_USE_MANTLE	Enable the Mantle endpoint. Set to <code>1</code> or <code>true</code> .
ANTHROPIC_BEDROCK_MANTLE_BASE_URL	Override the default Mantle endpoint URL
CLAUDE_CODE_SKIP_MANTLE_AUTH	Skip client-side authentication for proxy setups
ANTHROPIC_SMALL_FAST_MODEL_AWS_REGION	Override AWS region for the Haiku-class model (shared with Bedrock)

Troubleshooting

Authentication loop with SSO and corporate proxies

If browser tabs spawn repeatedly when using AWS SSO, remove the `awsAuthRefresh` setting from your **settings file**. This can occur when corporate VPNs or TLS inspection proxies interrupt the SSO browser flow. Claude Code treats the interrupted connection as an authentication failure, re-runs `awsAuthRefresh`, and loops indefinitely.

If your network environment interferes with automatic browser-based SSO flows, use `aws sso login` manually before starting Claude Code instead of relying on `awsAuthRefresh`.

Region issues

If you encounter region issues:

- Check model availability: `aws bedrock list-inference-profiles --region your-region`
- Switch to a supported region: `export AWS_REGION=us-east-1`
- Consider using inference profiles for cross-region access

If you receive an error “on-demand throughput isn’t supported”:

- Specify the model as an **inference profile** ID

Claude Code uses the Bedrock **Invoke API** and does not support the Converse API.

Mantle endpoint errors

If `/status` does not show `Amazon Bedrock (Mantle)` after you set `CLAUDE_CODE_USE_MANTLE`, the variable is not reaching the process. Confirm it is exported in the shell where you launched `claude`, or set it in the `env` block of your **settings file**.

A `403` from the Mantle endpoint with valid credentials means your AWS account has not been granted access to the model you requested. Contact your AWS account team to request access.

A `400` that names the model ID means that model is not served on Mantle. Mantle has its own model lineup separate from the standard Bedrock catalog, so inference profile IDs such as `us.anthropic.claude-sonnet-4-6` will not work. Use a Mantle-format ID, or enable **both endpoints** so Claude Code routes each request to the endpoint where the model is available.

Additional resources

- **Bedrock documentation**
- **Bedrock pricing**
- **Bedrock inference profiles**
- **Bedrock token burndown and quotas**
- **Claude Code on Amazon Bedrock: Quick Setup Guide**
- **Claude Code Monitoring Implementation (Bedrock)**